executes. If the decision is `deny`, the action is blocked, and the agent must revise its plan.

Conceptually, the flow looks like this:

```
build authorization request
send request to policy engine
if decision == permit:
    execute tool
else:
    return denial to agent and replan
```

This pattern transforms authorization from a static gate into a continuous control loop. Every action the agent attempts is evaluated in context, ensuring that dynamically generated behavior stays within defined boundaries.

> ***NOTE***
>
> Appendix C provides hands-on demonstrations of the authorization patterns described in this chapter. Section C.3 shows the reactive authorization model in action, illustrating how each tool invocation is evaluated by a policy enforcement point and how permit or deny decisions feed back into the agent's execution loop.

18.3.1 Authorization as part of the feedback loop

Besides preventing unsafe actions, embedding authorization in the agent loop also changes the agent's behavior.

When a traditional application encounters an authorization failure, it typically returns an error to the caller. In an agent system, a denial serves as input to the agent's reasoning process. The agent interprets the failure, updates its understanding of the environment, and attempts an alternative strategy.